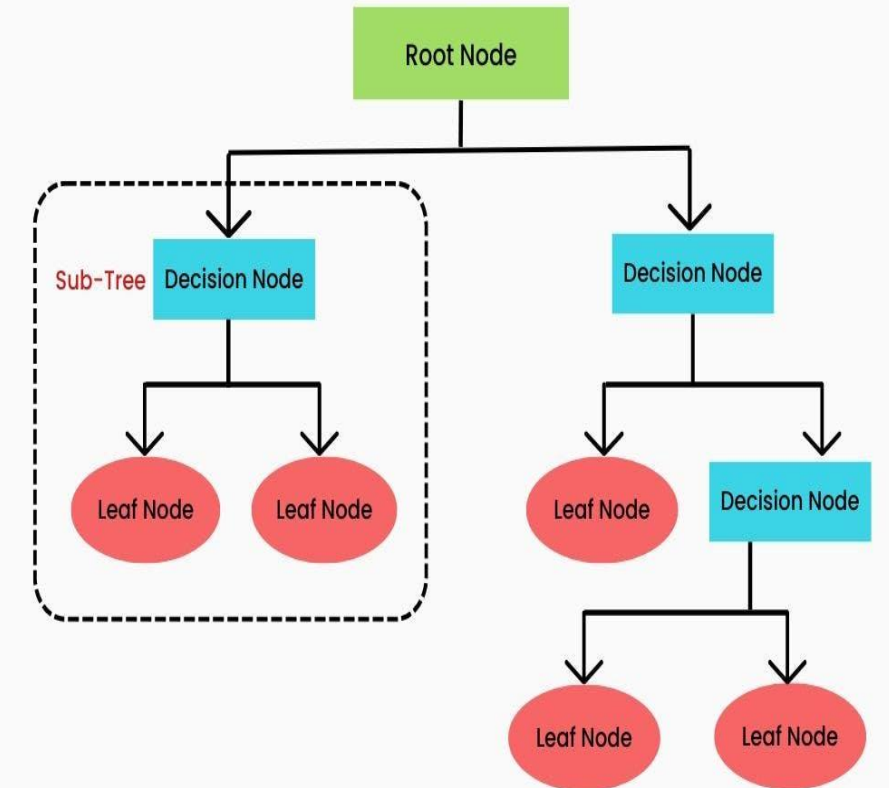


Decision Tree

Decision Tree Learning

- A decision tree is a supervised learning algorithm, which is utilized for both classification and regression tasks.
- It has a hierarchical, tree structure, which consists of a root node, branches, internal nodes and leaf nodes.
- Each node denotes a test on an attribute value, each branch represents an outcome of the test, and tree leaves represent classes or class distributions.
- Decision trees can easily be converted to classification rules.
- Learned trees can also be re-represented as sets of if-then rules to improve human readability.



Decision tree induction

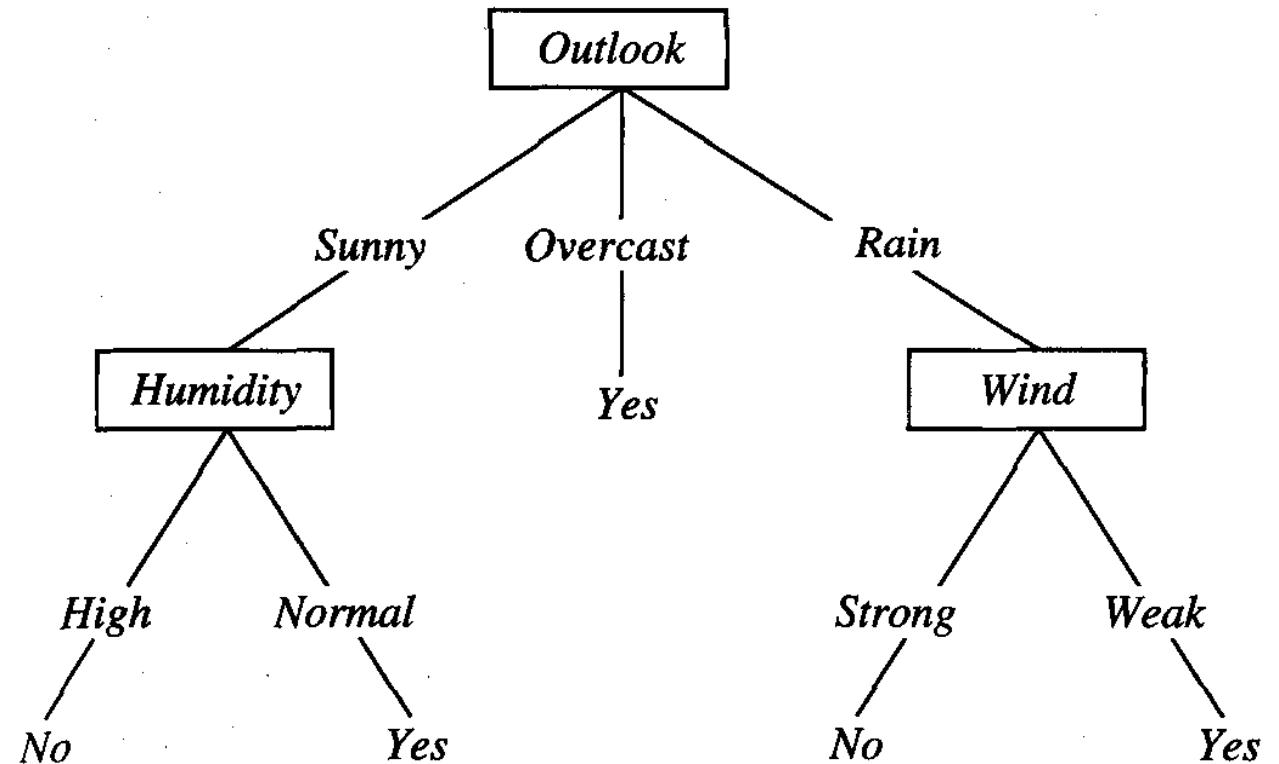
- Decision tree induction constructs a flowchart like structure where each internal (non leaf) node denotes a test on an attribute, each branch corresponds to an outcome of the test, and each external (leaf) node denotes a class label

Terminologies

- **Root Node** : It represents the entire population or sample and this further gets divided into two or more homogeneous sets
- **Leaf Node**: Node cannot be further segregated into further nodes
- **Parent/Child node**: Root node is the parent node and all the other nodes branched from it is know as child node
- **Branch/sub tree**: Formed by splitting the tree/node
- **Splitting** : It is dividing the root node/sub node into different parts on the basis of some condition.
- **Pruning**: Opposite of splitting, basically removing unwanted branches from the tree
- **Entropy**: Measure that tells the purity/impurity of samples
- **Information Gain**: It is the decrease in entropy after a dataset is split on the basis of an attribute. Constructing a decision tree is all about finding attribute that returns the highest information gain (Useful in deciding which attribute can be used as root node)
- **Reduction in variance**: It is an algorithm used for continuous target variables (regression problems). The split with lower variance is selected as the criteria to split the population.
- **Gini index**: the measure of purity or impurity used in building decision tree in CART ()

DECISION TREE REPRESENTATION

- A decision tree for the concept *PlayTennis*
- This tree classifies Saturday mornings according to whether or not they are suitable for playing tennis.



- Decision trees classify instances by sorting them down the tree from the root to some leaf node, which provides the classification of the instance.
- Each node in the tree specifies a test of some attribute of the instance, and each branch descending from that node corresponds to one of the possible values for this attribute.
- An instance is classified by starting at the root node of the tree, testing the attribute specified by this node, then moving down the tree branch corresponding to the value of the attribute in the given example.
- This process is then repeated for the subtree rooted at the new node.
- the decision tree shown in Figure corresponds to the expression

(Outlook = Sunny $\wedge$ Humidity = Normal)

$\vee$ (Outlook = Overcast)

$\vee$ (Outlook = Rain $\wedge$ Wind = Weak)

- Each path from the tree root to a leaf corresponds to a conjunction of attribute tests, and the tree itself to a disjunction of these conjunctions.
- **APPROPRIATE PROBLEMS FOR DECISION TREE LEARNING**
 - Instances are represented by attribute-value pairs
 - The target function has discrete output values
 - Disjunctive descriptions may be required.
 - The training data may contain errors
 - The training data may contain missing attribute values
- Decision tree learning has therefore been applied to problems such as learning to classify medical patients by their disease, equipment malfunctions by their cause, and loan applicants by their likelihood of defaulting on payments.

THE BASIC DECISION TREE LEARNING ALGORITHM

- Most algorithms that have been developed for learning decision trees are variations on a core algorithm that employs a top-down, greedy search through the space of possible decision trees. - ID3 algorithm (**Iterative Dichotomiser**).
- Basic algorithm, ID3, learns decision trees by constructing them top down, beginning with the question "which attribute should be tested at the root of the tree?"
- For this, each instance attribute is evaluated using a statistical test to determine how well it alone classifies the training examples.
- The best attribute is selected and used as the root node of the tree.
- A descendant of the root node is then created for each possible value of this attribute, and the training examples are sorted to the appropriate descendant node.
- The entire process is then repeated using the training examples associated with each descendant node to select the best attribute to test at that point in the tree.

Which Attribute Is the Best?

- We will define a statistical property, called **information gain**, that measures how well a given attribute separates the training examples according to their target classification.
- **Information gain** of an attribute, is the expected reduction in entropy caused by partitioning the examples according to the attribute A

$$Gain(A) = Info(D) - Info_A(D).$$

Where, $Info(D)$ is the **entropy** of collection D and $Info_A(D)$ is the **entropy** of attribute A in collection D

- **ENTROPY MEASURES**

- *Entropy*, is the (im)purity of an arbitrary collection of examples.
- Given a collection D , containing positive and negative examples of some target concept, the expected information (**entropy**) needed to classify a tuple in D is given by:

$$Info(D) = - \sum_{i=1}^m p_i \log_2(p_i),$$

where p_i is the nonzero probability that an arbitrary tuple in D belongs to class C_i and is estimated by $|C_{i,D}|/|D|$. A log function to the base 2 is used, because the information is encoded in bits. $Info(D)$ is just the average amount of information needed to identify the class label of a tuple in D . Note that, at this point, the information we have is based solely on the proportions of tuples of each class. $Info(D)$ is also known as the **entropy** of D .

- *Entropy is 0 if all members of S belong to the same class.*
- *Entropy is 1 when the collection contains an equal number of positive and negative examples.*
- *If the collection contains unequal numbers of positive and negative examples, the entropy is between 0 and 1.*

- Computation of $\text{Info}_A(D)$

- Suppose we were to partition the tuples in D on some attribute A having v distinct values, $\{a_1, a_2, \dots, a_v\}$, as observed from the training data.
- Attribute A can be used to split D into v partitions or subsets, $\{D_1, D_2, \dots, D_v\}$, where D_j contains those tuples in D that have outcome a_j of A . These partitions would correspond to the branches grown from node N .
- How much more information would we still need (after the partitioning) to arrive at an exact classification? This amount is measured by

$$\text{Info}_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \times \text{Info}(D_j).$$

Other Attribute Selection Measures

- Gain Ratio
- Gini Index

Decision Tree Algorithm

Algorithm: Generate_decision_tree. Generate a decision tree from the training tuples of data partition D .

Input:

- Data partition, D , which is a set of training tuples and their associated class labels;
- *attribute_list*, the set of candidate attributes;
- *Attribute_selection_method*, a procedure to determine the splitting criterion that “best” partitions the data tuples into individual classes. This criterion consists of a *splitting_attribute* and, possibly, either a *split point* or *splitting subset*.

Output: A decision tree.

Method:

- (1) create a node N ;
- (2) if tuples in D are all of the same class, C then
- (3) return N as a leaf node labeled with the class C ;
- (4) if *attribute_list* is empty then
- (5) return N as a leaf node labeled with the majority class in D ; // majority voting
- (6) apply *Attribute_selection_method*(D , *attribute_list*) to find the “best” *splitting_criterion*;
- (7) label node N with *splitting_criterion*;
- (8) if *splitting_attribute* is discrete-valued and
 multiway splits allowed then // not restricted to binary trees
- (9) *attribute_list* $\leftarrow$ *attribute_list* – *splitting_attribute*; // remove *splitting_attribute*
- (10) for each outcome j of *splitting_criterion*
 // partition the tuples and grow subtrees for each partition
- (11) let D_j be the set of data tuples in D satisfying outcome j ; // a partition
- (12) if D_j is empty then
- (13) attach a leaf labeled with the majority class in D to node N ;
- (14) else attach the node returned by *Generate_decision_tree*(D_j , *attribute_list*) to node N ;
- endfor
- (15) return N ;

Class-Labeled Training Tuples from the *AllElectronics* Customer Database

<i>RID</i>	<i>age</i>	<i>income</i>	<i>student</i>	<i>credit_rating</i>	<i>Class: buys_computer</i>
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

- Step 1: compute the expected information needed to classify a tuple in D

$$Info(D) = - \sum_{i=1}^m p_i \log_2(p_i),$$

$$Info(D) = -\frac{9}{14} \log_2\left(\frac{9}{14}\right) - \frac{5}{14} \log_2\left(\frac{5}{14}\right) = 0.940 \text{ bits.}$$

- Step 2: Compute the expected information requirement for each attribute

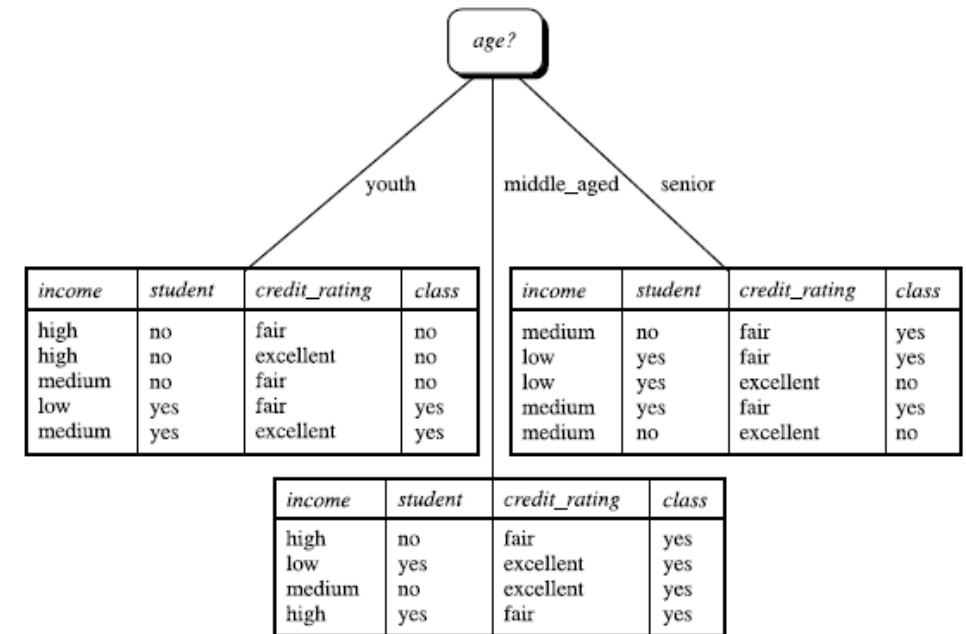
$$Info_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \times Info(D_j).$$

$$\begin{aligned} Info_{age}(D) &= \frac{5}{14} \times \left(-\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5}\right) \\ &\quad + \frac{4}{14} \times \left(-\frac{4}{4} \log_2 \frac{4}{4} - \frac{0}{4} \log_2 \frac{0}{4}\right) \\ &\quad + \frac{5}{14} \times \left(-\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5}\right) \\ &= 0.694 \text{ bits.} \end{aligned}$$

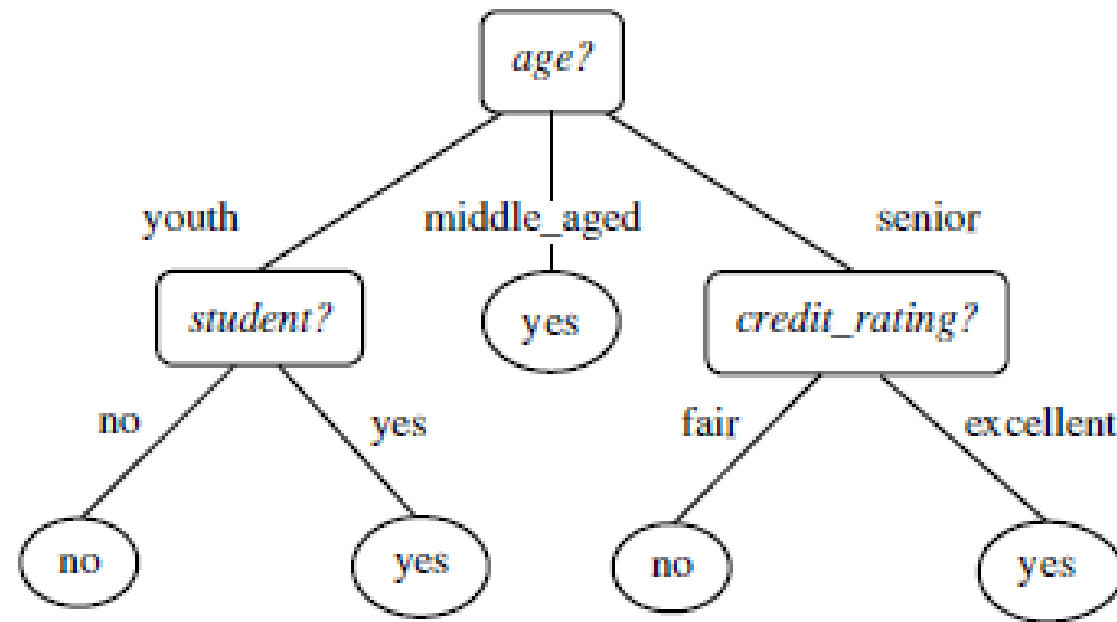
- Step 3: Compute Gain of the attributes:

$$Gain(age) = Info(D) - Info_{age}(D) = 0.940 - 0.694 = 0.246 \text{ bits.}$$

- Similarly, compute $Gain(income) = 0.029$ bits, $Gain(student) = 0.151$ bits, and $Gain(credit\ rating) = 0.048$ bits.
- Because age has the highest information gain among the attributes, it is selected as the splitting attribute.



e 6.5 The attribute *age* has the highest information gain and therefore becomes the splitting attribute at the root node of the decision tree. Branches are grown for each outcome of *age*. The tuples are shown partitioned accordingly.

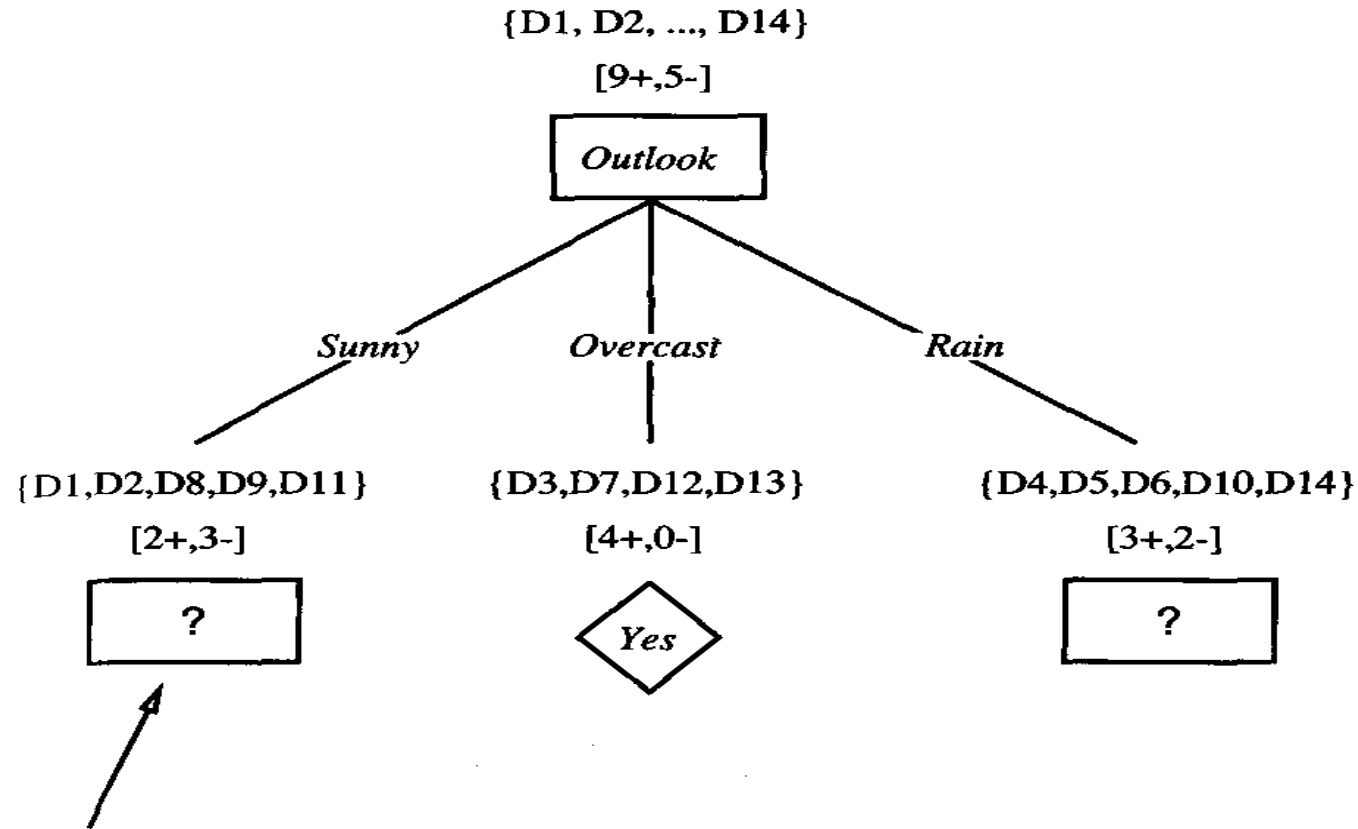


Example 2

- Training examples for the target concept ***PlayTennis***

Day	<i>Outlook</i>	<i>Temperature</i>	<i>Humidity</i>	<i>Wind</i>	<i>PlayTennis</i>
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

- Which attribute should be the root in the tree?
- ID3 determines the information gain for each candidate attribute (i.e., ***Outlook***, ***Temperature***, ***Humidity***, and ***Wind***), then selects the one with highest information gain.



Which attribute should be tested here?

$$S_{\text{sunny}} = \{D1, D2, D8, D9, D11\}$$

$$\text{Gain}(S_{\text{sunny}}, \text{Humidity}) = .970 - (3/5) 0.0 - (2/5) 0.0 = .970$$

$$\text{Gain}(S_{\text{sunny}}, \text{Temperature}) = .970 - (2/5) 0.0 - (2/5) 1.0 - (1/5) 0.0 = .570$$

$$\text{Gain}(S_{\text{sunny}}, \text{Wind}) = .970 - (2/5) 1.0 - (3/5) .918 = .019$$

Find which attribute should be the root in the tree?

Table 4.1 Medical Data

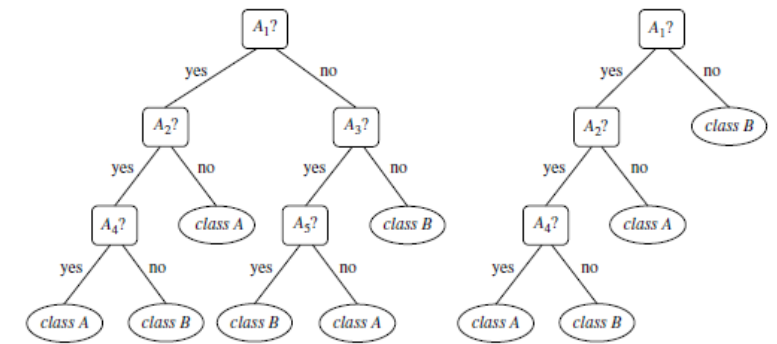
<i>o</i>	<i>Gender</i>	<i>Age</i>	<i>BP</i>	<i>Drug</i>
1	Male	20	Normal	A
2	Female	73	Normal	B
3	Male	37	High	A
4	Male	33	Low	B
5	Female	48	High	A
6	Male	29	Normal	A
7	Female	52	Normal	B
8	Male	42	Low	B
9	Male	61	Normal	B
10	Female	30	Normal	A
11	Female	26	Low	B
12	Male	54	High	A

Advantages of using decision trees:

- A decision tree does not need scaling of information.
- Missing values in data also do not influence the process of building a choice tree to any considerable extent.
- A decision tree model is automatic and simple to **explain** to the technical team as well as stakeholders.
- Compared to other algorithms, decision trees need less exertion for data preparation during pre-processing.
- A decision tree does not require a standardization of data.

Tree pruning

- When a decision tree is built, many of the branches will reflect anomalies in the training data due to noise or outliers.
- Tree pruning methods address this problem of *overfitting* the data. Such methods typically use statistical measures to remove the least reliable branches.
- Methods:
 - Pre pruning or Early stopping: Preventing the tree from growing too big or deep
 - Post Pruning: Allowing a tree to grow to its full depth and then getting rid of various branches based on various criteria
 - Ensembling or using averages of multiple models such as Random Forest



re 8.6 An unpruned decision tree and a pruned version of it.

- **Pre-pruning:**

- In the prepruning approach, a tree is “pruned” by halting its construction early (e.g., by deciding not to further split or partition the subset of training tuples at a given node).
- Upon halting, the node becomes a leaf. The leaf may hold the most frequent class among the subset tuples or the probability distribution of those tuples.
- If partitioning the tuples at a node would result in a split that falls below a prespecified threshold, then further partitioning of the given subset is halted.
- There are difficulties, however, in choosing an appropriate threshold. High thresholds could result in oversimplified trees, whereas low thresholds could result in very little simplification.

- ***Post Pruning***

- In this technique, we allow the tree to grow to its maximum depth. Then we remove parts of the tree to prevent overfitting.
- We effectively consider subtrees of the full tree which are evaluated on a criteria and then removed. Hence we are effectively going ‘up’ the tree and converting leaves to nodes and subtrees. The criteria whether a particular consolidation goes through or not is usually MSE for regression trees and classification error for classification trees.
- A challenge with post pruning is that a decision tree can grow very deep and large and hence evaluating every branch can be computationally expensive.